Ciurea Ana-Sorina 331CC

Tema 1 – Agregator de stiri

1. **Feedback:**

Aceasta tema a reprezentat un exercitiu util pentru intelegerea procesarii paralele a datelor (JSON) si a provocarilor impuse de sincronizarea firelor de executie.

Dificultati: Cea mai mare provocare a fost gestionarea corecta a formatarii stricte a fisierelor de iesire (pentru checker) si optimizarea accesului la resursele partajate pentru a evita *race condition*-urile fara a penaliza performanta prea tare.

Timp de lucru: Aproximativ 25-30 ore .

2. **Paralelizare:**

Etapele executiei:

Executia fiecarui WorkerThread este divizata in doua faze majore, separate de o bariera de sincronizare:

Etapa 1 (Map & Count):

- Fiecare thread citeste fisierele alocate si parseaza JSON-urile folosind biblioteca Jackson.
- Se populeaza doua harti concurente folosind operatii atomice (AtomicInteger). Aceasta etapa este necesara pentru a identifica duplicatele la nivel global inainte de procesarea finala.
- **Sincronizare:** Am utilizat CyclicBarrier pentru a garanta ca toate thread-urile au terminat de numarat aparitiile articolelor inainte de a trece la decizia de eliminare a duplicatelor.

Etapa 2 (Filter & Process):

- Fiecare thread itereaza din nou prin articolele incarcate local.
- **Filtrare:** Se consulta hartile globale
- **Agregare:** Articolele unice sunt adaugate in structuri thread-safe (ConcurrentHashMap pentru categorii/limbi, Collections.synchronizedList pentru lista globala).
- Procesarea cuvintelor cheie (tokenizare, eliminare stop-words) se face in paralel doar pentru articolele valide.

Mecanisme de sincronizare

- **ConcurrentHashMap:** Pentru stocarea datelor agregate (categorii, limbi, statistici) fara a bloca intregul map la scriere.
- **AtomicInteger:** Pentru incrementarea thread-safe a contoarelor (autori, cuvinte, duplicate).
- **CyclicBarrier:** Pentru separarea fazei de numarare de faza de procesare.
- **Bloc synchronized:** Folosit strict pentru actualizarea variabilei mostRecentArticle, fiind o zona critica mica unde se compara timestamp-uri.

3. Analiza de performanta si scalabilitate

Testele au fost rulate pe o masina personala cu urmatoarele specificatii performante:

Componenta	**Specificatii**
Model Laptop	Lenovo Yoga Pro 7 14IMH9
Procesor (CPU)	Intel Core Ultra 9 185H
Arhitectura CPU	16 nuclee (6 P-cores, 8 E-cores, 2 LPE-cores), 22 threads
Frecventa	Pana la 5.10 GHz (Max Turbo), 24MB Cache
Memorie RAM	32 GB LPDDR5X
Stocare	1 TB SSD (PCIe NVMe)
Sistem de Operare	Linux (WSL 2 pe Windows 11)
Java Version	OpenJDK 21 (sau versiunea instalata in WSL)

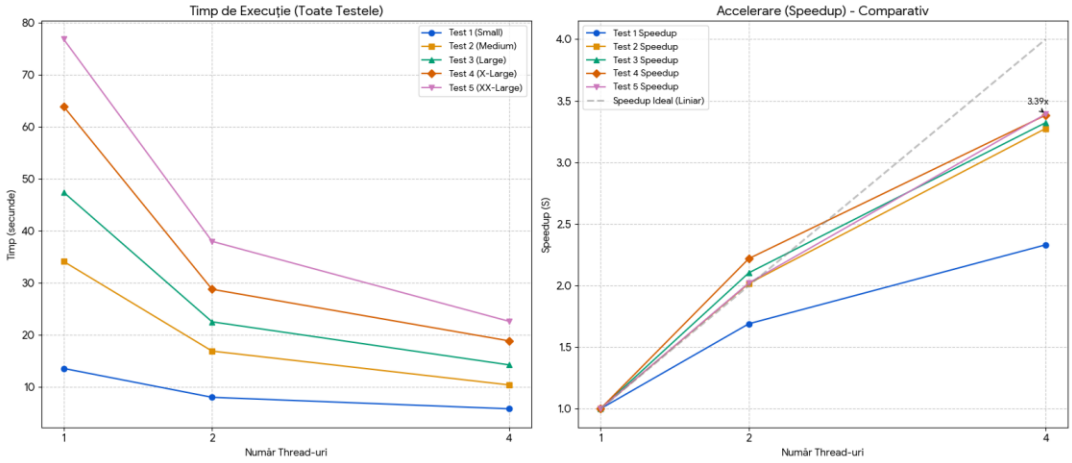
Am calculat eficienta folosind formula: $E(p) = S(p)/p$

Conform cerințelor, pentru a asigura acuratetea datelor si a elimina variatiile introduse de sistemul de operare, fiecare valoare de timp din tabelele de mai jos reprezintă **media aritmetică a 3 rulări consecutive**.

Testele prezentate (Test 1 - Test 5) corespund testelor temei.

Test	**Nr. Thread-uri (p)**	**Timp Mediu (Tp) [s]**	**Accelerație (Sp)**	**Eficiență (Ep)**
Test 1	**1**	**13.59s**	**1.00x**	**100%**

	2	**8.040s**	**1.69x**	**84.5%**
	4	**5.830s**	**2.33x**	**58.3%**
Test 2	**1**	**34.15s**	**1.00x**	**100%**
	2	**16.926s**	**2.01x**	**100.5%**
	4	**10.426s**	**3.27x**	**81.8%**
Test 3	**1**	**47.45s**	**1.00x**	**100%**
	2	**22.553s**	**2.10x**	**105.0%**
	4	**14.280s**	**3.32x**	**83.0%**
Test 4	**1**	**63.94s**	**1.00x**	**100%**
	2	**28.810s**	**2.21x**	**110.5%**
	4	**18.896s**	**3.38x**	**84.5%**
Test 5	**1**	**76.88s**	**1.00x**	**100%**
	2	**38.023s**	**2.02x**	**101.0%**
	4	**22.660s**	**3.39x**	**84.8%**



Interpretare grafice:

1. Analiza Timpului de Executie (Graficul din Stanga)

Impactul dimensiunii datelor: Graficul evidentiaza clar diferenta de magnitudine intre testele mici si cele mari. **Testul 5** porneste de la un timp secvential ridicat (~77 secunde), in timp ce **Testul 1** necesita doar ~13 secunde.

Comportamentul curbei: Pentru testele mari (Test 4 si Test 5), panta descendenta este extrem de abrupta la trecerea de la 1 la 2 si 4 fire de executie. Aceasta indica o paralelizare foarte eficienta: adaugarea de resurse computationale (thread-uri) reduce drastic timpul de asteptare. In schimb, pentru Testul 1, curba tinde sa se aplatizeze, sugerand ca timpul este dominat de overhead-ul de management al thread-urilor si nu de procesarea efectiva.

2. Analiza Acceleratiei / Speedup (Graficul din Dreapta)

Eficienta in functie de incarcare: Acest grafic demonstreaza vizual principiul conform caruia *eficienta paralelizarii creste odata cu dimensiunea problemei*.

Testele 4 si 5: Sunt cele mai performante. La 4 thread-uri, atingem un speedup de **~3.4x**, ceea ce reprezinta o eficienta excelenta de aproximativ 85%.

Testul 1 : Are cea mai mica accelerare (**2.33x** la 4 thread-uri)

Concluzie: Graficele valideaza scalabilitatea solutiei implementate. Programul se comporta robust pe seturi mari de date, unde overhead-ul de sincronizare devine neglijabil in raport cu volumul de munca, permitand o accelerare aproape liniara.